

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра №806 «Вычислительная математика и программирование»

Курсовая работа по курсу
“Дискретный анализ”

Персистентные структуры данных

Студент: Цирулев Николай Владимирович

Группа: М8О-308Б-22

Преподаватель: Макаров Никита Константинович

Оценка: _____

Дата: _____

Подпись: _____

Москва, 2024

Условие

Ограничение времени	5 секунд
Ограничение памяти	256 Mb
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Вам дан набор горизонтальных отрезков, и набор точек. Для каждой точки определите сколько отрезков лежит строго над ней.

Ваше решение должно работать online, то есть должно обрабатывать запросы по одному после построения необходимой структуры данных по входным данным. Чтение входных данных и запросов вместе и построение по ним общей структуры запрещено.

Формат ввода В первой строке вам даны два числа n и m ($1 \leq n, m \leq 10^5$) - количество отрезков и количество точек соответственно. В следующих n строках вам заданы отрезки, в виде троек чисел l, r и h ($-10^9 \leq l < r \leq 10^9, -10^9 \leq h \leq 10^9$) - координаты x левой и правой границ отрезка и координата y отрезка соответственно. В следующих m строках вам даны пары чисел x, y ($-10^9 \leq x, y \leq 10^9$) - координаты точек.

Формат вывода

Для каждой точки выведите количество отрезков над ней.

Метод решения

Немного теории:

Для решения задачи воспользуемся декартовым деревом для хранения отрезков.

Однако если хранить данные отрезков в обычном сбалансированном дереве поиска, то решение не пройдет чекер с ошибкой "Memory Limit", поэтому придется модифицировать наше дерево, используя такую концепцию, как "Персистентные структуры данных".

Пусть есть сбалансированное дерево поиска. Все операции в нем делаются за $O(h)$, где h — высота дерева, а высота дерева $O(\log(n))$, где n — количество вершин. Пусть необходимо сделать какое-то обновление в этом сбалансированном дереве, например, добавить очередной элемент, но при этом нужно не потерять старое дерево. Возьмем узел, в который нужно добавить нового ребенка. Вместо того чтобы добавлять нового ребенка, скопируем этот узел, к копии добавим нового ребенка, также скопируем все узлы вплоть до корня, из которых достигим первый скопированный узел вместе со всеми указателями. Все вершины, из которых измененный узел не достигим, мы не трогаем. Количество новых узлов всегда будет порядка логарифма (так как мы используем сбалансированное дерево поиска). В результате имеем доступ к обеим версиям дерева.

Теперь подытожим, персистентное дерево поиска (в нашем случае это декартово дерево), работающее по таким правилам, сохраняет все версии, доступные для чтения в любой момент, однако редактировать можно только последнюю версию кода.

Теперь применим теорию к нашему алгоритму:

В данной задаче нам нужно определить для точки, сколько отрезков лежит строго над ней. Поэтому логично будет хранить в массиве для каждого y список отрезков, которые выше данной координаты.

Интуитивное решение довольно просто: Создадим массив, посчитав что-то вроде массива префиксных сумм (i -ый элемент массива будет хранить не только те отрезки которые находятся на данной высоте, но и все те, что находятся выше). Далее пройдемся для каждой точки по этому массиву и найдем нужные отрезки.

Проблемы две: низкая скорость алгоритма и слишком большой размер используемой памяти. Повысить скорость можно использованием сбалансированного дерева поиска вместо линейного списка. Заметим, что если сравнить два соседних элемента нашего массива, то окажется, чтобы большая часть отрезков у них будут одинаковы. Для того, чтобы не хранить большое количество деревьев, будем хранить одно персистентное декартово дерево. В итоге у нас получается построение дерева за $O(n \log(n))$ и поиск за $O(m \log(n))$.

Описание программы

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <random>

template <typename T>
int BinarySearch(std::vector<T>& v, int x) {
    int l = -1;
    int r = v.size();
    if (x < v[0].x || x > v[v.size() - 1].x) {
        return -1;
    }
    while (l + 1 < r) {
        int mid = l + (r - l) / 2;

        if (v[mid].x <= x) {
            l = mid;
        } else {
            r = mid;
        }
    }
    return l;
}

struct TNode {
    TNode* left;
    TNode* right;
    int key;
    int priority;
    int count;
    int sumOfCounts;
};

int sumOfCounts(TNode* root) {
    if (root == nullptr) {
        return 0;
    }
    return root->sumOfCounts;
}
```

```

TNode* SearchInTreap(TNode* root, int key) {
    if (root == nullptr) {
        return root;
    } else if (root->key == key) {
        return root;
    } else if (key < root->key) {
        return SearchInTreap(root->left, key);
    } else {
        return SearchInTreap(root->right, key);
    }
}

std::pair<TNode*, TNode*> Split(TNode* root, int key) {
    if (root == nullptr) {
        return {nullptr, nullptr};
    } else if (key < root->key) {
        std::pair<TNode*, TNode*> leftTree = Split(root->left, key);
        TNode* rightTree = new TNode();
        rightTree->right = root->right;
        rightTree->key = root->key;
        rightTree->priority = root->priority;
        rightTree->count = root->count;
        rightTree->left = leftTree.second;
        rightTree->sumOfCounts = sumOfCounts(rightTree->right) + sumOfCounts(leftTree.first);
        return {leftTree.first, rightTree};
    } else {
        std::pair<TNode*, TNode*> rightTree = Split(root->right, key);
        TNode* leftTree = new TNode();
        leftTree->left = root->left;
        leftTree->key = root->key;
        leftTree->priority = root->priority;
        leftTree->count = root->count;
        leftTree->right = rightTree.first;
        leftTree->sumOfCounts = sumOfCounts(leftTree->left) + sumOfCounts(rightTree.second);
        return {leftTree, rightTree.second};
    }
}

TNode* InsertUtil(TNode* root, int key, int priority) {
    TNode* newRoot = new TNode();
    if (root == nullptr || priority > root->priority) {

```

```

        std::pair<TNode*, TNode*> splitted = Split(root, key);
        newRoot->left = splitted.first;
        newRoot->right = splitted.second;
        newRoot->key = key;
        newRoot->priority = priority;
        newRoot->sumOfCounts = sumOfCounts(splitted.first) + sumOfCounts(splitted.second);
        return newRoot;
    }
    newRoot->key = root->key;
    newRoot->priority = root->priority;
    newRoot->sumOfCounts = root->sumOfCounts + 1;
    newRoot->count = root->count;
    if (key < root->key) {
        newRoot->right = root->right;
        newRoot->left = InsertUtil(root->left, key, priority);
    } else {
        newRoot->left = root->left;
        newRoot->right = InsertUtil(root->right, key, priority);
    }
    return newRoot;
}

```

```

TNode* CopyNodesWithIncrement(TNode* root, int key) {
    TNode* newRoot = new TNode();
    if (root->key == key) {
        newRoot->left = root->left;
        newRoot->right = root->right;
        newRoot->key = key;
        newRoot->priority = root->priority;
        newRoot->count = root->count + 1;
        newRoot->sumOfCounts = root->sumOfCounts + 1;
        return newRoot;
    }
    newRoot->key = root->key;
    newRoot->priority = root->priority;
    newRoot->sumOfCounts = root->sumOfCounts + 1;
    newRoot->count = root->count;
    if (key < root->key) {
        newRoot->right = root->right;
        newRoot->left = CopyNodesWithIncrement(root->left, key);
    } else {
        newRoot->left = root->left;
    }
}

```

```

        newRoot->right = CopyNodesWithIncrement(root->right , key);
    }
    return newRoot;
}

TNode* Insert(TNode* root , int key , int priority) {
    if (!SearchInTreap(root , key)) {
        return InsertUtil(root , key , priority);
    }
    return CopyNodesWithIncrement(root , key);
}

void PrintTree(TNode* head , int space) {
    if (head == nullptr)
        return;
    space += 10;
    PrintTree(head->right , space);
    std::cout << '\n';
    for (int i = 10; i < space; ++i) {
        std::cout << ' ';
    }
    std::cout << head->key << " , " << head->count << '\n';
    PrintTree(head->left , space);
}

TNode* Merge(TNode* leftTree , TNode* rightTree) {
    if (leftTree == nullptr) {
        return rightTree;
    }

    if (rightTree == nullptr) {
        return leftTree;
    }

    TNode* newRoot = new TNode();
    if (leftTree->priority > rightTree->priority) {
        newRoot->key = leftTree->key;
        newRoot->priority = leftTree->priority;
        newRoot->left = leftTree->left;
        newRoot->count = leftTree->count;
        newRoot->right = Merge(leftTree->right , rightTree);
    }
}

```

```

        newRoot->sumOfCounts = sumOfCounts(newRoot->left) + sumOfCounts(newRoot->right);
    } else {
        newRoot->key = rightTree->key;
        newRoot->priority = rightTree->priority;
        newRoot->right = rightTree->right;
        newRoot->count = rightTree->count;
        newRoot->left = Merge(leftTree, rightTree->left);
        newRoot->sumOfCounts = sumOfCounts(newRoot->left) + sumOfCounts(newRoot->right);
    }
    return newRoot;
}

```

```

TNode* RemoveUtil(TNode* root, int key) {
    if (root->key == key) {
        return Merge(root->left, root->right);
    }
    TNode* newRoot = new TNode();

    newRoot->key = root->key;
    newRoot->priority = root->priority;
    newRoot->sumOfCounts = root->sumOfCounts - 1;
    newRoot->count = root->count;
    if (key < root->key) {
        newRoot->right = root->right;
        newRoot->left = RemoveUtil(root->left, key);
    } else {
        newRoot->left = root->left;
        newRoot->right = RemoveUtil(root->right, key);
    }
    return newRoot;
}

```

```

TNode* CountNodesWithDecrement(TNode* root, int key) {
    TNode* newRoot = new TNode();

    if (root->key == key) {
        newRoot->left = root->left;
        newRoot->right = root->right;
        newRoot->key = key;
        newRoot->priority = root->priority;
        newRoot->count = root->count - 1;
        newRoot->sumOfCounts = root->sumOfCounts - 1;
    }
}

```



```

        return newRoot;
    }
    newRoot->key = root->key;
    newRoot->priority = root->priority;
    newRoot->sumOfCounts = root->sumOfCounts - 1;
    newRoot->count = root->count;
    if (key < root->key) {
        newRoot->right = root->right;
        newRoot->left = CountNodesWithDecrement(root->left , key);
    } else {
        newRoot->left = root->left;
        newRoot->right = CountNodesWithDecrement(root->right , key);
    }
    return newRoot;
}

TNode* Remove(TNode* root , int key) {
    TNode* node = SearchInTreap(root , key);
    if (!node) {
        return root;
    }
    if (node->count > 0) {
        return CountNodesWithDecrement(root , key);
    }
    return RemoveUtil(root , key);
}

struct TPoint {
    int x;
    int y;
    bool flag;
    TNode* root = nullptr;
    bool operator<(const TPoint& other) const {
        if (x != other.x) {
            return x < other.x;
        } else {
            return flag > other.flag;
        }
    }
};

int GetResult(TNode* root , int y) {

```

```

    if (!root) {
        return 0;
    }
    if (root->key <= y) {
        return GetResult(root->right, y);
    } else {
        return sumOfCounts(root->right) + GetResult(root->left, y) + 1 + root->count;
    }
}

int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(0);
    std::random_device rd;
    std::mt19937 gen(rd());

    std::vector<TPoint> points;
    int n, m;
    std::cin >> n >> m;
    std::vector<int> priorities(2 * n);
    for (int i = 0; i <= 2 * n - 1; ++i) {
        priorities[i] = i;
    }
    std::shuffle(priorities.begin(), priorities.end(), gen);

    for (int i = 0; i < n; ++i) {
        int l, r, h;
        std::cin >> l >> r >> h;
        points.push_back({l, h, true, nullptr});
        points.push_back({r, h, false, nullptr});
    }
    std::sort(points.begin(), points.end());

    points[0].root = Insert(points[0].root, points[0].y, priorities[0]);
    for (int i = 1; i < 2 * n; ++i) {
        if (points[i].flag) {
            points[i].root = Insert(points[i-1].root, points[i].y, priorities[i]);
        } else {
            points[i].root = Remove(points[i-1].root, points[i].y);
        }
    }
    std::vector<TPoint> uniquePoints;

```

```

uniquePoints.push_back(points[2 * n - 1]);
for (int i = 2 * n - 2; i >= 0; i--) {
    if ((points[i].x != points[i + 1].x) || (points[i].flag != points[i + 1].flag))
        uniquePoints.push_back(points[i]);
}
std::reverse(uniquePoints.begin(), uniquePoints.end());

for (int i = 0; i < m; ++i) {
    int x, y;
    std::cin >> x >> y;
    int idx = BinarySearch(uniquePoints, x);
    if (idx == -1) {
        std::cout << 0 << '\n';
    } else {
        if (uniquePoints[idx].x == x && !uniquePoints[idx].flag) {
            --idx;
        }
        std::cout << GetResult(uniquePoints[idx].root, y) << '\n';
    }
}
}

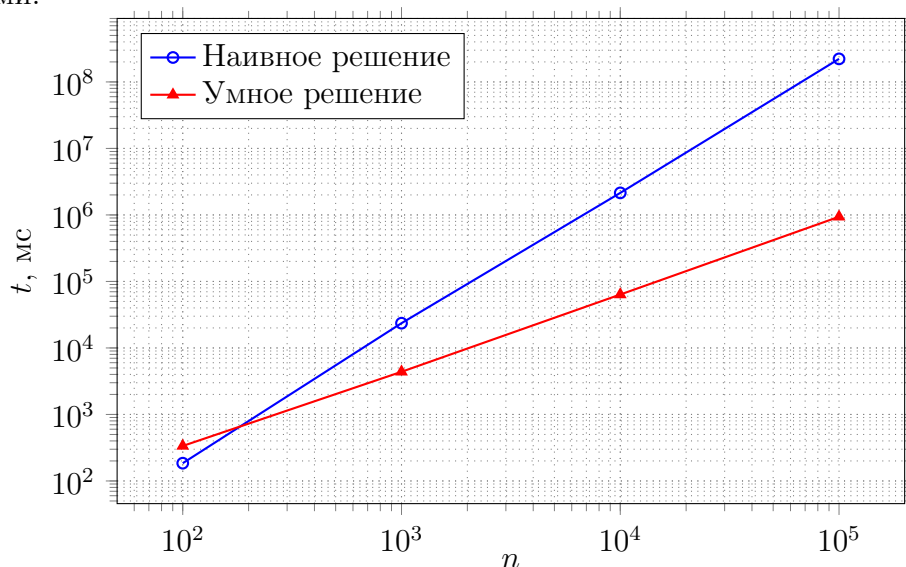
```

Тест производительности

Асимптотика написанного алгоритма - $O((n+m) \log(n))$. Для лучшей наглядности приведём таблицу, в которой время работы алгоритма сопоставляется с вводимыми данными и сравнивается с наивным решением. Для удобства расчетов сделаем $m = n$.

n	Время работы алгоритма (наивное решение), мс	Время работы алгоритма, мс
100	185	336
1000	23571	4380
10000	2145349	63562
100000	222067581	938645

Как мы видим по таблице, теоретическое время работы подтверждается реальными данными.



Ниже приведена программа `benchmark.cpp`, использовавшаяся для определения времени работы алгоритма:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <chrono>
#include <random>
#include "main.cpp"
```

```
vector<int> NaiveSolution(int n, int m, std::vector<TTriplet> segments, std:::
    std::vector<int> res;
    for(int i = 0; i < m; i++) {
        int count = 0;
```

```

        for(int j = 0; j < n; j++) {
            if (segments[j].l <= points[i].x && segments[j].r >= points[i].x)
                count++;
        }
        res.push_back(count);
    }
    return res;
}

std::vector<int> SmartSolution(int n, int m, std::vector<TTriplet> segments,
    std::random_device rd;
    std::mt19937 gen(rd()));

std::vector<TPoint> vec;
std::vector<int> priorities(2 * n);
for (int i = 0; i <= 2 * n - 1; ++i) {
    priorities[i] = i;
}
std::shuffle(priorities.begin(), priorities.end(), gen);

for (int i = 0; i < n; ++i) {
    vec.push_back({segments[i].l, segments[i].h, true, nullptr});
    vec.push_back({segments[i].r, segments[i].h, false, nullptr});
}
std::sort(vec.begin(), vec.end());

vec[0].root = Insert(vec[0].root, vec[0].y, priorities[0]);
for (int i = 1; i < 2 * n; ++i) {
    if (vec[i].flag) {
        vec[i].root = Insert(vec[i-1].root, vec[i].y, priorities[i]);
    } else {
        vec[i].root = Remove(vec[i-1].root, vec[i].y);
    }
}
std::vector<TPoint> unique;
unique.push_back(vec[2 * n - 1]);
for (int i = 2 * n - 2; i >= 0; i--) {
    if ((vec[i].x != vec[i + 1].x) || (vec[i].flag != vec[i + 1].flag)) {
        unique.push_back(vec[i]);
    }
}
}

```

```

std::reverse(unique.begin(), unique.end());
std::vector<int> res;
for (int i = 0; i < m; ++i) {
    int idx = BinarySearch(unique, points[i].x);
    if (idx == -1) {
        res.push_back(0);
    } else {
        if (unique[idx].x == points[i].x && !unique[idx].flag) {
            —idx;
        }
        res.push_back(GetResult(unique[idx].root, points[i].y));
    }
}
return res;
}

int main() {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> int_dist;
    for (int k = 100; k < 10e5; k *= 10) {
        std::cout << k << "_&_";
        std::vector<TTriplet> segments;
        std::vector<TPair> points;
        for (int i = 0; i < k; ++i) {
            TTriplet seg;
            TPair pnt;
            int d;
            seg.l = int_dist(gen);
            d = int_dist(gen);
            seg.r = seg.l + d;
            seg.h = int_dist(gen);
            pnt.x = int_dist(gen);
            pnt.y = int_dist(gen);
            segments.push_back(seg);
            points.push_back(pnt);
        }
        auto start1 = std::chrono::high_resolution_clock::now();
        std::vector<int> res1 = NaiveSolution(k, k, segments, points);
        auto finish1 = std::chrono::high_resolution_clock::now();
        auto duration1 = std::chrono::duration_cast<std::chrono::microseconds>
            std::cout << duration1.count() << "_&_";
    }
}

```


Выводы

В ходе выполнения данной работы были изучены методы построения персистентных структур данных. Также в результате выполнения курсовой работы было успешно реализовано персистентное декартово дерево. Алгоритм продемонстрировал высокую эффективность в задаче.

Персистентные структуры данных широко используются на практике, например, для эффективного хранения систем версионирования кода или для решения геометрических задач, подобных задаче в работе.

Знания, полученные в ходе выполнения работы, помогут в дальнейшей работе.